

Queues in Computer Science

Queues are a fundamental data structure in computer science, often compared to stacks but differing in their mechanism. While stacks operate on a **Last-In, First-Out (LIFO)** principle, **queues follow a First-In, First-Out (FIFO)** principle. This means the element inserted first is the first one to be removed. A common real-life analogy for a queue is a movie ticket counter, where the first person in line is the first to get a ticket.

1. Basic Concepts and Operations

A queue is typically represented as a line where elements are added at one end and removed from the other.

- **Front (or Head):** The end from which elements are removed.
- **Rear (or Tail):** The end where new elements are added.

Primary Operations:

- **Add (Enqueue):** Inserts an element at the rear.
- **Remove (Dequeue):** Removes the element from the front.
- **Peek:** Returns the element at the front without removing it.

Auxiliary Operations:

- `size()`: Returns the number of elements in the queue.
 - `isEmpty()`: Checks if the queue is empty.
-

2. Prerequisites

Before learning queues, you should be familiar with: - Arrays - Linked Lists - Stacks - Recursion

3. Built-in Queue (Java Collection Framework)

In Java, `Queue` is an interface, and you can use implementations like `ArrayDeque` or `LinkedList`.

```
Queue<Integer> q = new ArrayDeque<>(); // or new LinkedList<>();
```

Built-in Operations:

- `add(element)`: Inserts at the rear.
- `remove()`: Removes and returns the head. Throws exception if empty.

- `poll()` : Removes and returns the head. Returns `null` if empty.
 - `element()` : Returns head without removing. Throws exception if empty.
 - `peek()` : Returns head without removing. Returns `null` if empty.
 - `size()` : Returns number of elements.
 - `isEmpty()` : Checks if queue is empty.
-

4. Queue Implementation using Arrays

Key Components:

- `int[] arr` : Underlying array.
- `int front` : Index of the first element.
- `int rear` : Index of the last element.
- `int size` : Current number of elements.

Challenges:

- **Fixed size**: Cannot grow dynamically.
- **Wasted space**: Removed elements leave unused spaces.

Logic:

- **add(value)**: Place at `rear`, adjust pointers, increment size.
 - **remove()**: Remove from `front`, adjust pointers, decrement size.
 - **peek()**: Return `arr[front]`.
 - **display()**: Iterate from `front` to `rear`.
-

5. Queue Implementation using Linked Lists

Key Components:

- `Node head` : Points to front.
- `Node tail` : Points to rear.
- `int size` : Tracks number of elements.

Advantages:

- Unlimited size (dynamic).
- $O(1)$ enqueue and dequeue.

Disadvantages:

- Extra memory for node pointers.

Logic:

- **add(value):** Insert new node at `tail`.
 - **remove():** Remove node at `head`.
 - **peek():** Return `head.data`.
 - **display():** Traverse nodes from head to tail.
-

6. Circular Queue

A circular queue solves the wasted space problem in arrays by making `rear` wrap around to the beginning.

Logic:

- **add(value):** Insert at `rear`, wrap around if needed.
 - **remove():** Remove from `front`, wrap around if needed.
 - **peek():** Return `arr[front]`.
 - **display():** Handle both normal and wrapped-around cases.
-

7. Deque (Double-Ended Queue)

A **Deque** allows insertion and deletion at both ends.

Built-in Deque in Java:

```
Deque<Integer> dq = new ArrayDeque<>();
```

Operations:

- **Adding:** `addFirst()`, `addLast()`, `add()`
- **Removing:** `removeFirst()`, `removeLast()`, `pollFirst()`, `pollLast()`
- **Accessing:** `getFirst()`, `getLast()`, `peekFirst()`, `peekLast()`

Implementation:

Typically implemented with a **doubly linked list**.

Summary

Queues operate on FIFO principle and can be implemented using arrays, linked lists, or circular arrays. Java provides built-in support via `Queue` and `Deque` interfaces. Advanced forms like circular queues and deques extend functionality, solving limitations of basic queues.